

This homework is due on Sunday, March 9, 2025, at 10:59PM.

1. Understanding Dropout (Coding Question)

In this question, you will analyze the effect of dropout in a simplified setting. Please follow the instructions in `q_coding_dropout.ipynb` `dropout.ipynb` and answer the questions in your submission of the written assignment. The notebook does not need to be submitted.

- (a) (No dropout, least-square) **The mathematical expression of the OLS solution, and the solution calculated in the code cell.**
- (b) (No dropout, gradient descent) **The solution in the code cell. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?**
- (c) (Dropout, least-square) **The solution in the code cell.**
- (d) (Dropout, gradient descent) **Describe the shape of the training curve. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?**
- (e) (Dropout, gradient descent, large batch size) **Describe the loss curve and compare it with the loss curve in the last part. Why are they different? Also compare the trained weights with the one calculated by the least-square formula.**
- (f) Refer back to the cells you ran in part (e). **Analyze how and why adding dropout changes the following: (i) How large were the final weights w_1 and w_2 compared to each other. (ii) How large the contribution of each term (i.e. $10w_1 + w_2$) is to the final output. Why does this change occur?** (This does not need to be a formal math proof).
- (g) (Optional) Sweeping over the dropout rate **Fill out notebook section (G).** You should see that as the dropout rate changes, w_1 and w_2 change smoothly, except for a discontinuity when dropout rates are 0. Explain this discontinuity.
- (h) (Optional) Optimizing with Adam: Run the cells in part (H). **Does the solution change when you switch from SGD to Adam? Why or why not?**
- (i) Dropout on real data: **Run the notebook cells in part (I), and report on how they affect the final performance.**

2. Debugging DNNs

- (a) Your friends want to train a classifier for a new app they're designing. They implement a deep convolutional network and train models with two configurations: a 20 layer model and a 56 layer model. However, they observe the following training curves and are surprised that the 20-layer network has better training as well as test error.

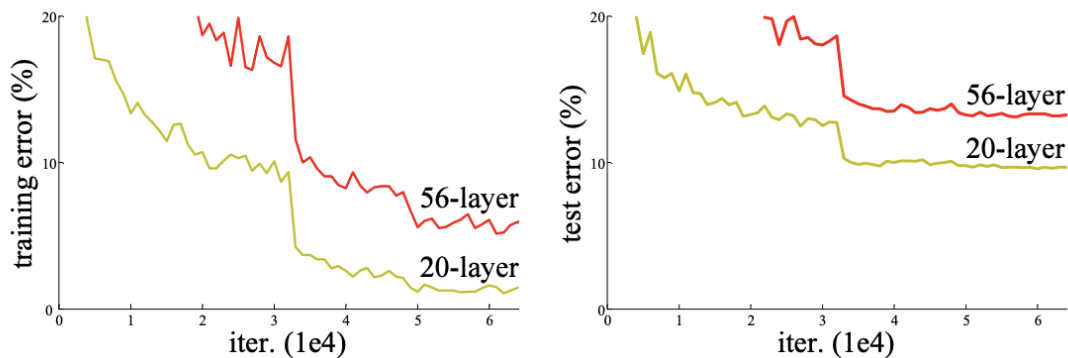


Figure 1: Training deep networks on CIFAR10

What are the potential reasons for this observation? Are there changes to the architecture design that could help mitigate the problem?

- (b) You and your teammate want to compare batch normalization and layer normalization for the ImageNet classification problem. You use ResNet-152 as a neural network architecture. The images have input dimension $3 \times 224 \times 224$ (channels, height, width). You want to use a batch size of 1024; however, the GPU memory is so small that you cannot load the model and all 1024 samples at once — you can only fit 32. Your teammate proposes using a gradient-accumulation algorithm:

Gradient accumulation refers to running the forward and backward pass of a model a fixed number of steps (`accumulation_steps`) without updating the model parameters, while aggregating the gradients. Instead, the model parameters are updated every (`accumulation_steps`). This allows us to increase the effective batch size by a factor of `accumulation_steps`.

You implement the algorithm in PyTorch as:

```
model.train()
optimizer.zero_grad()
for i, (inputs, labels) in enumerate(training_set):
    predictions = model(inputs)
    loss = loss_function(predictions, labels)
    loss = loss / accumulation_steps
    loss.backward()
    if (i+1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Note that the `.backward()` operator in PyTorch implicitly keeps accumulating the gradient for all the parameters, unless zero'd out with an `optimizer.zero_grad()` call.

Before running actual experiments, your friend suggests that you should test whether the gradient accumulation algorithm is implemented correctly. To do so, you collect the output logits (i.e. the outputs of the last layer) from two models — ResNet-152, one with batchnorm and the other with layernorm — using different combinations of batch sizes and the number of accumulation steps that keep the effective batch size to 32.

[Note that the effective batch size is product of the batch size and `accumulation_steps`. In other words, the possible combinations for effective batch-size 32 are:

$(batch_size, accumulation_steps) = (1, 32), (2, 16), (4, 8), (8, 4), (16, 2), (32, 1).$]

Here, the (32,1) combination is the baseline approach without the “gradient accumulation” trick, and we want to see whether the others output exactly the same logits (up to floating point inaccuracies) as the baseline.

On running these tests, you observe that one of models (either with batchnorm or with layernorm), doesn’t pass the test. **Which one is it, and why?**

- (c) You are training a CIFAR model and observe that the model is diverging (instead of the training loss decreasing over iterations). **Debug the pseudocode and give a correction that you believe would actually result in reasonable convergence during training.**

(Note: You can assume that the datasets are loaded correctly, model is trained with SGD optimizer with learning rate= 0.001, batchsize= 100)

(HINT: Ideas from the previous part of this question might be relevant.)

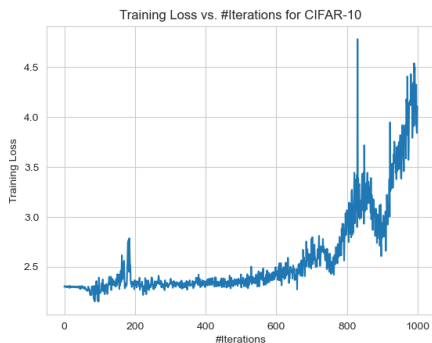


Figure 2: Training loss for CIFAR10

```
model.train()
optimizer.zero_grad()
for (inputs, labels) in training_set:
    predictions = model(inputs)
    loss = loss_fn(predictions, labels)
    loss.backward()
    optimizer.step()
```

3. Optimization Techniques for “Bad” Objective Functions

In this coding question, you will learn about three cool techniques that can help you optimize challenging objective functions that are hard to optimize with vanilla gradient descent.

For the purpose of understanding these techniques, we will focus on local minima in this problem. Note that in general, local optima are not really a practical issue on most supervised learning objectives in modern deep learning. This is because as running stochastic gradient descent with these objective functions on overparameterized (large enough) neural networks often converge to a global optimum (there are often many optima due to symmetry in neural networks). If you are interested in some theoretical justifications, you might find this paper interesting ([Gradient Descent Finds Global Minima of Deep Neural Networks](#)).

However, for objective functions that are less standard (e.g., feedback signal from human) and in reinforcement learning, local minima and other things that manifest similarly can play a huge role. This coding question teaches you some basics on how you might be able to optimize these functions.

In particular, we will be working with the following objective function $f(x, y)$ and the goal is to find the best pair of (x, y) that maximizes the function.

Implement all the TODOs in the [opt.ipynb](#). **Answer the written questions below.**

- (a) (Part 1) **What do you observe from the optimization trajectory of your (x, y) parameters? Where do your parameters converge to?** Try a few different initialization values and see how the convergence changes.

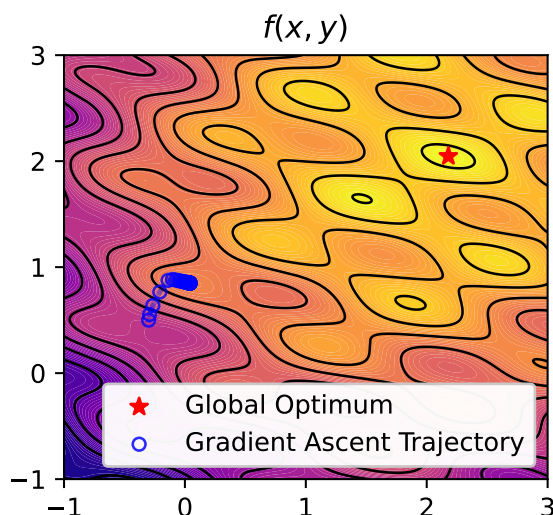


Figure 3: The objective function $f(x, y)$ has many local optima causing the naive gradient ascent approach to find a local maximum far away from the global optimum.

- (b) (Part 1) **What patterns do you observe from the basins of attraction visualization? How do they relate to the sine and cosine in our objective function?**
- (c) (Part 1) **What do you notice when the learning rate η changes? Is there a good learning rate setting that finds the global optimum better? Is there any other way that we can do to modify our gradient descent algorithm to overcome the local optimum issue?**
- (d) (Part 2) Take a close look at the global optimum for the original function (red star) and the global optimum for the smoothed function (blue star). **What do you observe as σ increases? Why? Can you give an example where the global optimum of the smoothed function is very far from the global optimum of the original function?**
- (e) (Part 3a) **What do you observe when you change δ , η (learning rate) and N (the sample size for Monte-Carlo estimate)?**
- (f) (Part 3b) **Can you provide an intuitive explanation of why the gradient estimator works? (Hint: which direction is the gradient estimator trying to push (x, y) into? By what magnitude?)**
- (g) (Part 3b) **What do you observe when you change η (learning rate) and N (the sample size for Monte-Carlo estimate)? How does it compare to the finite-different method?**
- (h) (Part 3c) Compare the trajectory of reparam gradient vs. policy gradient. **What do you observe? Is one more straight than the other one? Why do you think that is the case?**
- (i) (Part 4) **How well do you expect naive gradient descent to perform on this quantized objective function? Explain why.**
- (j) (Part 4) **Can you use reparameterization gradient for this function? Explain why.**

4. Policy Gradient and the Reparameterization Gradient Estimator

Do this question after the coding question above for a better understanding of the context.

In this question, you are going to derive two gradient estimators for the following objective function:

$$\mathcal{F}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\theta}} [f(\mathbf{x})], \quad (1)$$

where $\mathbf{x}$ is a random variable that follows the probability distribution of $p_\theta : \mathcal{X} \mapsto \Delta(\mathcal{X})$ that is parameterized by $\theta \in \mathbb{R}^P$, and $f : \mathcal{X} \mapsto \mathbb{R}$ is a function.

- (a) In the related coding question, you saw a special case where $\mathbf{x}$ is a k -D multivariate Gaussian random variable that is parameterized by the mean $\mu \in \mathbb{R}^K$ under the distribution,

$$p_\mu(\mathbf{x}) = (2\pi\sigma)^{-n/2} \exp\left(-\|\mathbf{x} - \mu\|_2^2 / (2\sigma^2)\right), \quad (2)$$

where $\sigma \in \mathbb{R}$ is a scalar constant. **Express $\nabla_\mu \mathcal{F}(\mu)$ as an expectation under $p_\mu(\mathbf{x})$** (i.e., find $g(\mathbf{x})$ in $\nabla \mathcal{F}(\mu) = \mathbb{E}_{\mathbf{x} \sim p_\mu} [g(\mathbf{x})]$). (Hint: $\mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x})] = \int p_\theta(\mathbf{x}) f(\mathbf{x}) d\mathbf{x}$)

- (b) **Rewrite the expression in $\mathcal{F}(\mu)$ with an expectation with the distribution over a standard normal $\mathcal{N}(0, \mathbf{I})$?** (Hint: if $\mathbf{x}$ follows the distribution of $p_\mu(\mathbf{x})$, then $\mathbf{x} - \mu$ follows the distribution of $\mathcal{N}(0, \sigma^2 \mathbf{I})$)
- (c) Using the expression you obtained from the previous part, **can you express $\nabla_\mu \mathcal{F}(\mu)$ in a similar way** (i.e., with an expectation with the distribution over $p_{\mu=0}(\mathbf{x})$)? (Hint: ∇_μ can be safely moved inside the expectation because the expectation no longer follows a distribution that depends on μ and expectations are linear.)

For the following two parts, we are going to generalize our findings from the specific Gaussian case to arbitrary p_θ 's (you may assume p_θ has well-defined gradient $\nabla_\theta p_\theta$ that expresses what change in parameters would make a particular realization more likely).

- (d) In the general case, **can you still write $\nabla \mathcal{F}(\theta)$ as an expectation under $p_\theta(\mathbf{x})$?** (Hint: look at your answer in Part (a) and see how the term inside your expectation relates to $\log p_\theta(\mathbf{x})$).
- (e) Assume there exists a function $g(\mathbf{z}, \theta) : \mathcal{Z} \times \mathbb{R}^P \mapsto \mathcal{X}$ and a distribution over $\mathbf{z}$, $p(\mathbf{z})$, such that $g(\mathbf{z}, \theta)$ has the same distribution as $\mathbf{x} \sim p_\theta(\mathbf{x})$. **Prove that $\nabla_\theta \mathcal{F}(\theta) = \mathbb{E}_{\mathbf{z} \sim \mathcal{Z}} [\nabla_\theta g(\mathbf{z})^\top \nabla_{\mathbf{x}} f|_{\mathbf{x}=g(\mathbf{z}, \theta)}]$.** In the previous part (b) and (c), we actually prove a special case of this. **Can you determine what g and $p(\mathbf{z})$ are for the special case?**

These kinds of tricks are practically useful because the spirit of stochastic gradient descent plays nicely whenever we have gradients that we can view as expectations.

5. Directed and Undirected Graphs

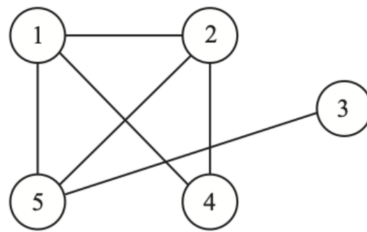


Figure 4: Simple Undirected Graph

Figure 4 shows a simple undirected graph whose adjacency matrices we want to make sure you can write down. Generally, an unnormalized adjacency matrix between the nodes of a directed or undirected graph is given by:

$$A_{i,j} = \begin{cases} 1 & \text{if there is an edge between node } i \text{ and node } j, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

This will be a symmetric matrix for undirected graphs. For a directed graph, we have:

$$A_{i,j} = \begin{cases} 1 & \text{if there is an edge from node } i \text{ to node } j, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

This need not to be symmetric for a directed graph, and is in fact typically not a symmetric matrix when we are thinking about directed graphs (otherwise, we'd probably be thinking of them as undirected graphs).

Similarly, the degree matrix of an undirected graph is a diagonal matrix that contains information about the degree of each vertex. In other words, it contains the number of edges attached to each vertex and it is given by:

$$D_{i,j} = \begin{cases} \deg(v_i) & \text{if } i = j, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

where the degree $\deg(v_i)$ of a vertex counts the number of times an edge terminates at that vertex.

For directed graphs, the degree matrix could be *In-Degree* when we count the number of edges coming into a particular node and *Out-Degree* when we count the number of edges going out of the node. We'll use the terms in-degree matrix or out-degree matrix to make it clear which one we are invoking.

Sometimes, imbalanced weights may undesirably affect the matrix spectrum (eigenvalues and eigenvectors). This occurs when a vertex with a large degree results in a large diagonal entry in the Laplacian matrix dominating the matrix properties. To solve that issue, a normalization scheme is applied which aims to make the influence of such vertices more equal to that of other vertices, by dividing the entries of the Adjacency matrix by the vertex degrees.

In that sense, a normalized adjacency matrix is given by:

$$A^{Normalized} = AD^{-1} \quad (6)$$

and a symmetrically normalized adjacency matrix is given by

$$A^{SymNorm} = D^{-1/2}AD^{-1/2} \quad (7)$$

Additionally, the Laplacian matrix relates many useful properties of a graph. In fact, the spectral decomposition of the Laplacian matrix of a graph allows for the construction of low-dimensional embeddings that appear in many machine learning applications. In other words, there is a relation between the properties of a graph and the spectra (eigenvalues and eigenvectors) of matrices associated with the graph, such as its adjacency matrix or Laplacian matrix.

Given a simple graph G with n vertices $v_1, \dots, v_n$, its unnormalized Laplacian matrix $L_{n \times n}$ is defined element-wise as:

$$L_{i,j} = \begin{cases} \deg(v_i) & \text{if } i = j, \\ -1 & \text{if } i \neq j \text{ and } v_i \text{ is adjacent to } v_j, \\ 0 & \text{otherwise.} \end{cases} \quad (8)$$

or equivalently by the matrix:

$$L = D - A \quad (9)$$

where D is the degree matrix and A is the adjacency matrix of the graph.

We could also compute the symmetrically normalized Laplacian which is inherited from the adjacency matrix normalization scheme as shown below:

$$L^{SymNorm} = I - A^{SymNorm} \quad (10)$$

where I is the identity matrix, A is the unnormalized adjacency matrix, and L is the unnormalized Laplacian.

- (a) **Show that $L^{SymNorm}$ could also be written as:**

$$L^{SymNorm} = D^{-1/2} L D^{-1/2} \quad (11)$$

where D is the degree matrix, and L is the unnormalized Laplacian.

- (b) **Write the unnormalized adjacency A , the degree matrix, D , and the symmetrically normalized adjacency matrix, $A^{SymNorm}$, of the graph in Figure. 4.**
- (c) **Write the symmetrically normalized Laplacian matrix of the graph in Figure. 4.**
- (d) **Compute A^2 , A^3**

We now want to estimate the traffic flow of inner downtown Berkeley and we know the road network shown below. The goal of the estimation is to estimate the traffic flow on each road segment. The flow estimates should satisfy the conservation of vehicles exactly at each intersection as indicated by the arrows.

The intersections are labeled a to h. The road segments are labeled 1 to 22. The arrows indicate the direction of traffic.

Hint: think about the best way to represent the road network in terms of matrices, vectors, etc.

- (e) **Write the unnormalized adjacency matrix of the graph in Figure 5.**
- (f) **Write the In-degree D_{in} and Out-degree D_{out} matrix of the graph in Figure. 5.**
- (g) **Write both of the symmetrically normalized In-degree and Out-degree Laplacian matrix of the graph in Figure. 5.**
- (h) **[Optional]** It is good to read <https://arxiv.org/pdf/1609.02907.pdf> and <https://distill.pub/2021/understanding-gnns/> to learn about the importance of the Adjacency and Laplacian matrices in graph representation.

6. The power of the graph perspective in clustering (Coding)

Implement all the TODOs in the [q_graph_clustering.ipynb](#). Answer the written questions below.

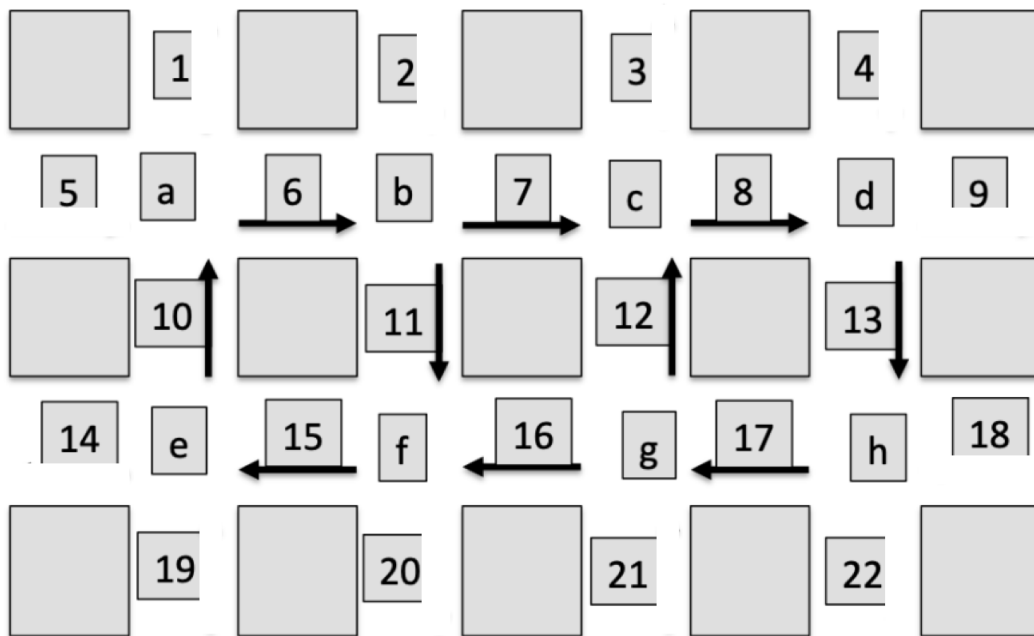


Figure 5: Simple Directed Graph

- (a) We used the KMeans algorithm implementation of sklearn, and showed our attempt to cluster this dataset into 3 classes. **Comment on the output the KMeans algorithm. Did it work? If so, explain why, if not, explain why not.**
- (b) `adjacency_matrix = ?`
- (c) As given, the data points in our dataset are represented simply with their 2D Cartesian coordinates. Let's now interpret every single point as a node in a graph. Our goal is to find a way to relate every node in the graph in such way that the points that are closer together and points that are far apart maintain that relationship explicitly.

That is, we will choose to look at every point in the dataset as a vertex in a graph where the edge connection between two vertexes is determined by the weighted distances between them.

In the notebook, implement a function that takes in the input dataset and some coefficient gamma and returns the adjacency matrix A :

$$A_{i,j} = e^{-\gamma \|x_i - x_j\|^2} \quad (12)$$

where x_i and x_j represent each point in the provided dataset, γ is positive. You may find the distance module from `scipy.spatial` useful.

Is this a directed or an undirected graph?

- (d) The degree matrix of an undirected graph is a diagonal matrix which contains information about the degree of each vertex. In other words, it contains the number of edges attached to each vertex and it is

given by:

$$D_{i,j} = \begin{cases} \deg(v_i) & \text{if } i = j, \\ 0 & \text{otherwise.} \end{cases}$$

where the degree $\deg(v_i)$ of a vertex counts the number of times an edge terminates at that vertex. Note that in the traditional definition of the adjacency matrix, this boils down to the diagonal matrix in which element along the diagonals are column-wise sum of the adjacency matrix.

Using the same idea, **write a function that takes in the adjacency matrix as an argument and returns the degree matrix.**

- (e) Using $\gamma = 7.5$, **compute the adjacency matrix A , degree matrix D and the symmetrically normalized adjacency matrix matrix M ,**

$$M = A^{SymNorm} = D^{-1/2} A D^{-1/2} \quad (13)$$

Note that another interpretation of the matrix M is that it shows the probability of moving/jumping from one node to another.

- (f) Applying SVD decomposition on M , **write a function that selects the top 3 vectors (corresponding to the highest singular values) in the matrix U and performs the same KMeans clustering used above on them. Show the plots. What do you observe? Did it work? If so, explain why, if not, explain why not.**

Intuition: By selecting the top 3 vectors of the U matrix, we are selecting a new representation of the data points which could be seen as a construction of a low dimension embedding of the data points as mentioned in problem 3.

- (g) Now let's think of the symmetrically normalized adjacency matrix obtained above as the transition matrix in of a Markov Chain. That is, it represents the probability of jumping from one node to another. In order to fully interpret M in such way, it needs to be a proper stochastic matrix which means that the sum of the elements in each column must add up to 1. **Write a function that takes in the matrix M and returns M_{stoch} , the stochastic version of M ; compute the stochastic matrix.**

Using SVD decomposition on the newly obtained stochastic matrix M_{stoch} , **use your function in part (e) to select the top 3 vectors of the matrix U_{stoch} and perform the same KMeans clustering used above on them and show the plots. What do you observe? Did it work?**

- (h) Now, let's investigate how we could have made the matrix M work directly in our original interpretation. To do this, normalize those 3 vectors before performing the clustering.

Show the plots. What do you observe? Did it work? If so, explain why normalizing the vectors gives what is expected.

Hint: you may use

```
from sklearn.preprocessing import normalize
```

7. Zachary's Karate Club (Coding)

Zachary's Karate Club (ZKC) is a social network of a university karate club, described in the paper "An Information Flow Model for Conflict and Fission in Small Groups" by Wayne W. Zachary.

A social network captures 34 members of a karate club, documenting links between pairs of members who interacted outside the club.

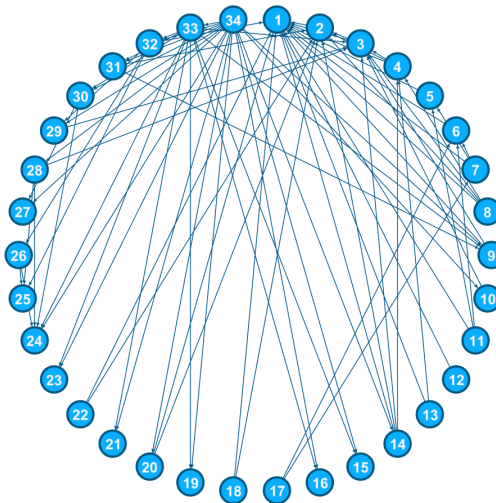


Figure 6: Zachary's Karate Club Graph

During the study a conflict arose between the officer/ administrator ("John A") and the instructor "Mr. Hi", which led to the split of the club into two.

Half of the members formed a new club around Mr. Hi; members from the other part found a new instructor or gave up karate.

Based on collected data Zachary correctly assigned all but one member of the club to the groups they actually joined after the split. You could read more about it here <https://www.jstor.org/stable/3629752>, and here https://commons.wikimedia.org/wiki/File:Social_Network_Model_of_Relationships_in_the_Karate_Club.png

We will train a GNN to cluster people in the karate club in such that people who are more likely to associate with either the officer or Mr. Hi will be close together, while the distance between the 2 classes will be far.

In the original paper titled "Semi-Supervised Classification with Graph Convolutional Networks" that can be found here <https://arxiv.org/pdf/1609.02907.pdf>, the authors framed this as a node-level classification problem on a graph. We will pretend that we only know the affiliation labels for some of the nodes (which we'll call our training set) and we'll predict the affiliation labels for the rest of the nodes (our test set).

Implement all the TODOs in `zkc.ipynb` and include your notebook with your submission.

- Go through `q_zkc.ipynb`. We want our network to be aware of information about the nodes themselves instead of only the neighborhood, so we add self loops our adjacency matrix. The **paper** called this $\tilde{A}$. **Compute $\tilde{A}$ to add self loops to your adjacency matrix.**
- Write a function that takes in $\tilde{A}$ as argument and returns the $\tilde{A}^{SymNorm}$ adjacency matrix.**
- The other input to our GNN is the graph node matrix X which contains node features. For simplicity, we set X to be the identity matrix because we don't have any node features in this example. **Generate the feature input matrix X .**
- We will now implement a single layer GNN. **Implement the forward and backward pass functions for `GNN_Layer` class.** Details can be found in the notebook.
- Run the forward and backward passes and ensure the checks pass.**

- (f) We are now ready to setup our classification network! **Use the GNN and Softmax layers to setup the network.**
- (g) **Instantiate the GNN model with the correct input and output dimensions.**
- (h) With the model, data and optimizer ready, **fill in the todos in the training loop function and train your model. Plot the clustered data.**
- (i) **Explain why we obtain 100% on accuracy on our test set, yet we see in the plot that 2 samples seem to be misclassified.**

8. Homework Process and Study Group

Citing sources and collaborators are an important part of life, including being a student!

We also want to understand what resources you find helpful and how much time homework is taking, so we can change things in the future if possible.

- (a) **What sources (if any) did you use as you worked through the homework?**
- (b) **If you worked with someone on this homework, who did you work with?**
List names and student ID's. (In case of homework party, you can also just describe the group.)
- (c) **Roughly how many total hours did you work on this homework?**

Contributors:

- Olivia Watkins.
- Anant Sahai.
- Jake Austin.
- Linyuan Gong.
- Saagar Sanghavi.
- Suhong Moon.
- Kumar Krishna Agrawal.
- Qiyang Li.
- Ryan Scott.
- Ria Doshi.
- Jerome Quenum.
- Anrui Gu.
- Matthew Lacayo.
- Past EECS 282 and 227 Staff.
- Romil Bhardwaj.